

# HealthcareAgent

---

HealthcareAgent is a FastAPI backend for managing prescriptions, lab reports, clinical notes, vitals, user metadata, and patient-facing recommendations. It supports bearer-token authentication, PostgreSQL persistence, UUID-based user-facing IDs, summaries, reminders, and a health-history-aware chat experience.

## Main Capabilities

- Doctor inputs for prescriptions, clinical notes, and vitals.
- Laboratory inputs for patient-linked lab reports.
- Patient history views with clinician- and patient-facing summaries.
- Patient-facing recommendations and reminders based on history and vitals.
- Context-aware chat grounded in a patient's medical history.

## Database

Copy `.env.example` to `.env` and set your local database settings. At minimum, set `POSTGRES_PASSWORD` for Docker and `DATABASE_URL` for the app:

```
POSTGRES_PASSWORD=your_password
DATABASE_URL=postgresql+psycopg2://postgres:postgres@localhost:5432/healthcareagent
```

If you are not using a `.env` file, set `DATABASE_URL` in your shell before running the app:

```
set
DATABASE_URL=postgresql+psycopg2://postgres:postgres@localhost:5432/healthcareagent
```

To run PostgreSQL locally with Docker:

```
docker compose up -d postgres
```

Then start the app:

```
uvicorn main:app --reload
```

On startup, the app tries to create the required tables automatically.

# Authentication

The API exposes two user roles: **patient** and **doctor**.

- **POST** `/api/auth/register` creates a user and returns a bearer token.
- **POST** `/api/auth/login` returns a bearer token for an existing user.
- **GET** `/api/auth/me` returns the authenticated user profile.
- **PUT** `/api/auth/me` updates the authenticated user profile.

Registration accepts optional profile fields for **sex**, **contact\_number**, and **emergency\_number**.

Send the token in the **Authorization: Bearer <token>** header for protected routes.

## API

The main endpoints are mounted under `/api`:

- **GET** `/api/patients` doctor-only patient list with profile, chief complaint, alerts, and last update.
- **POST** `/api/add-prescription`
- **GET** `/api/get-prescription?patient_id=<uuid>`
- **POST** `/api/add-clinical-note`
- **GET** `/api/get-clinical-note?patient_id=<uuid>`
- **POST** `/api/add-lab-report`
- **GET** `/api/get-lab-report?patient_id=<uuid>&report_date=...`
- **POST** `/api/add-vitals`
- **GET** `/api/get-vitals?patient_id=<uuid>&recorded_at=...`
- **GET** `/api/get-vitals-history?patient_id=<uuid>`
- **PUT** `/api/update-vitals/{patient_id}/{recorded_at}`
- **DELETE** `/api/delete-vitals/{patient_id}/{recorded_at}`
- **POST** `/api/summarize` with `{ "patient_id": "<uuid>" }`
- **GET** `/api/recommend` for the logged-in patient.
- **GET** `/api/remind` for the logged-in patient.
- **POST** `/api/talk` with patient-side conversation memory.

## Quick Start

1. Create a Python virtual environment and install dependencies:

```
python -m venv .venv
source .venv/Scripts/activate    # Windows PowerShell:
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

2. Create an `.env` file in the project root and set the required environment variables.

3. Run the app locally:

```
uvicorn main:app --reload  
# or, with Docker  
docker compose up --build
```

#### 4. Run tests:

```
pytest
```

## Project Layout

- Server entrypoint: [main.py](#)
- Requirements: [requirements.txt](#)
- Docker compose: [docker-compose.yml](#)
- Backend code: [backend/](#)
- Tests: [tests/](#)

## Development Tips

- Inspect the router and service modules under [backend/routers/](#) and [backend/services/](#) to see available endpoints and business logic.
- Use `uvicorn main:app --reload` for iterative development.

## Contributing

- Open an issue or PR describing the change.
- Keep changes small and focused; add tests for new behaviors.